

ATME COLLEGE OF ENGINEERING
DEPARTMENT OF CSE-DATA SCIENCE

LABORATORY REPORT
OF
PARALLEL PROGRAMMING(BDS701)

Submitted by

Name:

USN:

Faculty Incharge

Dr. Vinod Kumar P
Associate Professor

ATME COLLEGE OF ENGINEERING

13th KM Stone, Bannur Road, Mysore - 560 028

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

(DATA SCIENCE)

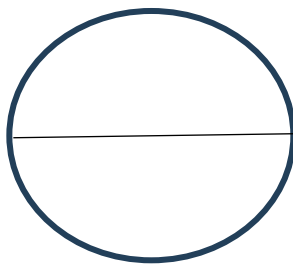
CERTIFICATE

This is to certify that Mr./ Mrs.

Bearing seat University Number

Has satisfactorily completed the laboratory Experiments in the course
titled.....

.....in theSemester of B.E Program
during the academic year 2025-26 As prescribed by the Visveswaraya Technological University
Belagavi,



Signature of the Staff

Signature of the HOD

INDEX

[illegible]

Sl No.	Experiments	Date	Page No	Marks
Average:				
Internal Lab Marks: _____ + _____ = _____ (LCR) (TEST)				
Signature of Staff ATME College of Engineering				

Introduction

Parallel processing is the use of multiple processors to divide a large task into smaller parts, execute those parts simultaneously, and then combine the results to solve a problem more quickly and efficiently.

This approach significantly reduces computation time and is crucial for complex problems in fields such as scientific simulations, AI, and cloud computing. The brain also uses parallel processing to simultaneously interpret various sensory inputs like sights, sounds, and smells, allowing it to form a comprehensive understanding of its environment.

How parallel processing works

Parallel processing divides a task between two or more microprocessors. Typically, a complex task is divided into multiple parts using a specialized software tool that assigns each part to a processor based on the task's component elements. Larger tasks are broken into multiple smaller parts that are appropriate for the number, type and size of available processing units. Then, each processor completes its part, and the software tool reassembles the data and executes the task.

Typically, each processor operates normally and performs operations in parallel as instructed, pulling data from the computer's memory. Processors also rely on software to communicate with each other so they can stay in sync concerning changes in data values. Assuming all the processors remain in sync with one another, at the end of a task, the software fits all the data pieces together.

Parallel processes are either *fine-grained* or *coarse-grained*. In fine-grained parallel processing, tasks communicate with one another multiple times. This is suitable for processes that require real-time data. Coarse-grained parallel processing, on the other hand, deals with larger pieces of a task and requires less frequent communication between processors. This type of processing is more useful in applications that have tasks with minimal interdependencies and can be naturally divided into larger parts.

Computers without multiple processors can also still be used in parallel processing if they're networked together to form a cluster.

Difference between serial and parallel processing

Parallel processing can complete multiple tasks using two or more processors whereas serial processing -- also called *sequential processing* -- only completes one task at a time using one processor. If a computer needs to complete multiple assigned tasks, it will complete one task at a time. Likewise, if a computer using serial processing needs to complete a complex task, it will take longer compared to a parallel processor.

History of parallel processing

In the earliest computers, only one program ran at a time. It would take a total of two hours to run both a computation-intensive program that would take one hour to run and a tape copying program that took one hour to run. An early form of parallel processing enabled the interleaved execution of both programs together. The computer would start an input/output (I/O) operation, and while it was waiting for the operation to complete, it would execute the

processor-intensive program. The total execution time for the two jobs was a little over one hour.

The next improvement was multiprogramming in the late 1950s and early 1960s. In a multiprogramming system, multiple programs submitted by users were each allowed to use the processor for a short time. To users, it appeared that all the programs were executing at the same time. Problems of resource contention first arose in these systems. Explicit requests for resources led to the problem of deadlock, where simultaneous requests for resources would effectively prevent a program from accessing the resource. Competition for resources on machines with no tie-breaking instructions led to the critical section routine.

Vector processing was another attempt, starting in 1964, to increase performance by doing more than one thing at a time. In this case, capabilities were added to machines to allow a single instruction to add or subtract, multiply or otherwise manipulate two arrays of numbers. This was valuable in certain engineering applications where data naturally occurred in the form of vectors or matrices. In applications with less well-formed data, vector processing wasn't as valuable.

The next step in parallel processing was the introduction of multiprocessing in the 1970s. In these systems, two or more processors shared the work to be done. The earliest versions had a primary and secondary configuration. The primary processor was programmed to be responsible for all the work in the system; the secondary processor performed only those tasks it was assigned by the primary processor. This arrangement was necessary because it wasn't then understood how to program the machines so they could cooperate in managing the resources of the system. Multiprocessing continued to improve into the 1980s, 1990s and beyond.

The early 2000s saw the development of distributed computing, often involving multiple independent computers that could work on a task together, without shared memory.

The early 2000s and onwards also saw an increase in the use of graphics processing units (GPUs), which consist of a large number of smaller processing units, called cores. Each core can execute its instructions independently of the others, which enables parallel processing. GPUs are also increasingly used for general-purpose computing tasks -- beyond just graphics.

Since then, there have been developments such as specialized hardware accelerators, which include field-programmable gate arrays and application-specific integrated circuits that speed up processing for specific parallel processing tasks.

Most recently, organizations have started to implement quantum computers that focus on creating quantum parallel processors that perform computations with qubits.

Benefits:

It increases processing speed and efficiency by handling multiple operations simultaneously.

Applications:

It's essential for high-performance computing, big data analysis, artificial intelligence, financial modeling, and other computationally intensive tasks.

Types:

Data Parallelism: A large dataset is split, and the same operation is performed on each chunk simultaneously.

Task Parallelism: Different parts of a program or task are executed independently in parallel.

Hardware types: Systems can use multiple CPU cores, or even multiple computer systems (parallel computing) to achieve parallelism.

Advantages of parallel processing

- **Increased speed and efficiency:** By executing multiple tasks at once, parallel processing dramatically reduces the time required to solve complex problems and increases system throughput.
- **Scalability:** As problems grow larger or more complex, parallel systems can be scaled up by adding more processors to handle the increased workload.
- **Improved problem-solving:** Parallel computing allows for the modeling and simulation of more complex, real-world phenomena, such as climate and financial models, that would be infeasible with sequential processing.
- **Cost-effectiveness:** Utilizing multiple, less expensive processors can be more economical than relying on a single, high-performance one.

Disadvantages of parallel processing

- **Programming complexity:** Writing programs that can run efficiently in parallel is more difficult than writing sequential programs and often requires specialized skills.
- **Synchronization issues:** Managing communication and synchronization between multiple processors can be complex, potentially leading to errors and reduced performance.
- **Higher hardware costs and power consumption:** Multicore and massively parallel systems require more hardware and consume more power, which can lead to higher operational costs.
- **Communication overhead:** The need for processors to exchange data with each other can create overhead, potentially limiting the performance gains, especially if the tasks are not well-suited for parallelization.

This lab manual provides an overview of:

- The Linux environment and commonly used commands.
- Basics of OpenMP (for shared-memory parallel programming).
- Basics of MPI (for distributed-memory parallel programming).

It is intended for students beginning with **parallel and high-performance computing** in lab environments.

Linux

Linux is a family of open-source operating systems that are Unix-like, with the Linux kernel at their core, created by Linus Torvalds in 1991. It is known for being secure, stable, and versatile, found in everything from smartphones and supercomputers to cloud platforms and everyday appliances. Linux is popular among developers for its flexibility and open-source nature, which allows for modification and redistribution.

What is Linux used for?

Linux is a popular platform for organisations hosting data, services, or applications securely. You can also use its desktop environment on personal computers. Common Linux use cases include:

- Server applications: Linux is suitable for running web servers, file servers, database servers, and other shared servers that require high-volume support.
- Software development: Linux's open-source software development tools are popular among enterprise-level businesses. The widely used version control system Git was developed by creator Linus Torvalds for Linux Kernel.
- Cloud computing: Linux's open-source, highly scalable nature makes it a suitable OS for cloud infrastructure.
- Game development: Linux can be installed on various platforms, including mobile devices and video game consoles.

Linux components

Operating systems comprise a suite of programs that enable a computer to operate. A Linux-based operating system consists of three main parts:

- the kernel,
- shell, and
- programs.

Take a look at a brief overview of each one.

What is a Linux kernel?

The Linux kernel is the central component of the Linux operating system. It serves as the communication interface between a computer's hardware and processes. Linux kernels are responsible for controlling all major functions and managing computer resources such as files, processes, memory, and devices.

What is a Linux shell?

A shell is a specialised program that enables users to execute commands and use operating system services. The Linux shell translates natural-language user commands and files into input that the kernel can understand. A Linux shell has two classification levels:

- **Command line shell:** A command line shell is accessible via a command line interface (CLI). CLIs are computer programs that rely on the direction of text-based input for task execution. Examples of command line interfaces include Terminal and Command Prompt.
- **Graphical shell:** A graphical shell is accessible via a graphical user interface (GUI). GUIs are interfaces designed for users to interact with by clicking, scrolling, tapping, or initiating other actions.

What are Linux programs?

Linux programs are typically designed to do one thing efficiently. These specialised programs can be used together in many different ways. You can think of them as customisation parts that enable you to add functionalities to your Linux operating system as needed.

Part I – Linux Environment & Commands

1.1 Understanding the Linux Environment

Linux is an open-source operating system widely used in servers, HPC clusters, and development environments. It offers:

- Multi-user capability
- Multitasking support
- Security and permissions
- Powerful command-line tools

1.2 Essential Linux Commands

File and Directory Commands

Command	Description	Example
ls	List directory contents.	ls
cd	Change directory.	cd /path/to/directory
pwd	Show current directory.	pwd
mkdir	Create a new directory.	mkdir new_directory
rmdir	Remove an empty directory.	rmdir empty_directory
rm	Delete files or directories.	rm file.txt
touch	Create an empty file.	touch new_file.txt
cp	Copy files or directories.	cp file.txt /path/to/destination
mv	Move or rename files.	mv file.txt /path/to/new_location
cat	Display file contents.	cat file.txt
nano / vim	Edit files in terminal.	nano file.txt
find	Search for files in a directory hierarchy.	find . -name "file.txt"
grep	Search text using patterns.	grep "pattern" file.txt

tar	Archive and compress files.	tar -cvf archive.tar file1.txt file2.txt
df	Show disk usage of file systems.	df
du	Show directory/file size.	du -sh /path/to/directory
chmod	Change file permissions.	chmod 755 file.txt
chown	Change file owner.	chown user:group file.txt
mount	Mount a filesystem.	mount /dev/sdb1 /mnt
umount	Unmount a filesystem.	umount /mnt

Networking Commands

Command	Description	Example
ps	Show running processes.	ps aux
htop	Enhanced version of top.	htop
kill	Send a signal to a process.	kill <PID>
killall	Kill processes by name.	killall <process name>
uptime	System uptime and load.	uptime
whoami	Current logged-in user.	whoami
env	Display environment variables.	env
strace	Trace system calls of a process.	strace -p <PID>
systemctl	Manage systemd services.	systemctl status <service_name>
journalctl	View system logs.	journalctl -xe
free	Display memory usage.	free -h
vmstat	Report virtual memory statistics.	vmstat 1
iostat	Report CPU and I/O statistics.	iostat
lsof	List open files by processes.	lsof
dmesg	Print kernel ring buffer messages.	dmesg

User and Permission Management Commands

Command	Description	Example
passwd	Change user password.	passwd <username>
adduser / useradd	Add a new user.	adduser <username> or useradd <username>
deluser / userdel	Delete a user.	deluser <username> or userdel <username>
usermod	Modify user account.	usermod -aG <group> <username>
newgrp	Log in to a new group.	newgrp <group>
id	Display user identity information.	id <username>
groups	Show group memberships.	groups <username>
sudo	Execute commands as root.	sudo <command>

File Transfer and Synchronization Commands

Command	Description	Example
curl	Transfer data from or to a server.	curl -O http://example.com/file.zip
wget	Download files from the web.	wget http://example.com/file.zip
sftp	Securely transfer files using SSH File Transfer Protocol.	sftp user@remote:/path/to/file

ftp	Transfer files using the File Transfer Protocol.	ftp ftp.example.com
scp	Securely copy files over SSH.	scp user@remote:/path/to/file /local/destination
rsync	Efficiently sync files and directories.	rsync -avz /local/directory/ user@remote:/path/to/destination

Text Processing Commands

Command	Description	Example
awk	Pattern scanning and processing.	awk '{print \$1}' file.txt
sed	Stream editor for filtering/modifying text.	sed 's/old/new/g' file.txt
cut	Remove sections from lines of text.	cut -d':' -f1 /etc/passwd
sort	Sort lines of text.	sort file.txt
grep	Search for patterns in text.	grep 'pattern' file.txt
wc	Count words, lines, and characters.	wc -l file.txt
join	Join lines of two files on a common field.	join file1.txt file2.txt
head	Output the first part of files.	head -n 10 file.txt
tail	Output the last part of files.	tail -n 10 file.txt

Shell Utilities and Shortcuts Commands

Command	Description	Example
history	Show previously entered commands.	history
alias	Create shortcuts for commands.	alias ll='ls -la'
unalias	Remove an alias.	unalias ll
clear	Clear the terminal screen.	clear
reboot	Reboot the system.	reboot
shutdown	Power off the system.	shutdown now
date	Display or set the system date and time.	date
sleep	Delay for a specified amount of time.	sleep 5
echo	Display a line of text.	echo "Hello, World!"
time	Measure the duration of command execution.	time ls
watch	Execute a program periodically, showing output fullscreen.	watch -n 5 df -h

Lab Exercise 1 – Linux Basics

1. Create a directory named lab1.
2. Inside it, create a file hello.txt with some text.
3. Copy the file to backup.txt.
4. Change permissions so only you can read/write the file.
5. Use grep to search for a word inside it.
6. Delete the backup file.

Part II – OpenMP Basics

Introduction to Parallel Programming with OpenMP in C/C++

2.1 Introduction

OpenMP (Open Multi-Processing) is an Application Programming Interface (API) that supports multi-platform shared-memory multiprocessing programming in C, C++, and Fortran. It is a widely used standard for parallel computing on systems with shared memory architectures.

Parallel programming is the process of breaking down a large task into smaller sub-tasks that can be executed simultaneously, thus utilizing the available computing resources more efficiently. OpenMP is a widely used API for parallel programming in C++. It allows developers to write parallel code easily and efficiently by adding simple compiler directives to their existing code.

It allows developers to write programs that can run efficiently on multi-core and multi-threaded systems by using compiler directives, libraries, and environment variables.

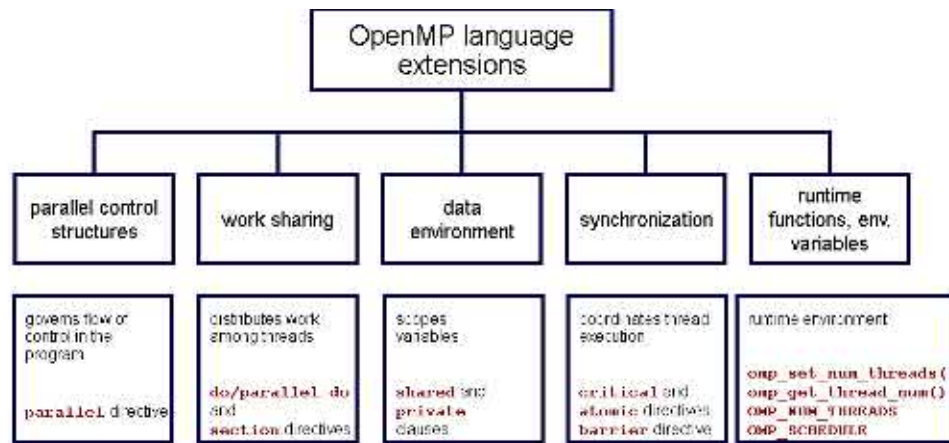
The primary purpose of OpenMP is to enable the creation of parallel applications to improve performance and reduce execution time on shared-memory architectures.

What is OpenMP

OpenMP is an implementation of multithreading, a method of parallelizing whereby a master thread (a series of instructions executed consecutively) forks a specified number of slave threads and the system divides a task among them. The threads then run concurrently, with the runtime environment allocating threads to different processors.

The core elements of OpenMP are the constructs for thread creation, workload distribution (work sharing), data-environment management, thread synchronization, user-level runtime routines and environment variables.

In C/C++, OpenMP uses `#pragmas`. Those instructions are comments to the non openmp compiler but can be understood if the compiler support them and if the compilation line includes instructions to include openmp in the final object. In Fortran OpenMP uses comments like “`!$omp`”, they are also comments in Fortran 90+ but can be used by the compiler if the compilation arguments are included.



Syntax of OpenMP

OpenMP uses compiler directives to indicate the parallel sections of the code. The directives are preceded by the "#pragma" keyword and take the form:

```
#pragma omp <directive> [clause[,clause]...]
```

Parameters

1. The following are some common OpenMP directives:

- "parallel": create a team of threads that execute the enclosed code block in parallel.
- "for": splits a loop into smaller iterations that can be executed in parallel by different threads.
- "sections": split the enclosed code block into different sections that can be executed in parallel.
- "single": specifies that a code block should be executed by only one thread.
- "critical": specifies that a code block should be executed by only one thread at a time.
- "atomic": specifies that a variable should be accessed atomically.

2. clause: The clauses provide additional information to the directives. For example, the "num_threads" clause specifies the number of threads to be used for a parallel section.

Steps for Parallel Programming

- Steps needed to achieve (openMP) parallelize in your programming:
- 1. Include the OpenMP header file:
`#include <omp.h>`
- 2. Add the OpenMP directives to the relevant sections of your code.
`#pragma omp parallel`

```
{
```

```
// Code block to be executed in parallel

}
```

Runtime Library functions

there are a number of functions that can be called from the openmp library. This is summary of some of them:

routine	Purpose
omp_set_num_threads	sets the number of threads that will be used in the next parallel region
omp_get_num_threads	returns the number of threads that are currently in the team executing the parallel region from which it is called
omp_get_max_threads	returns the maximum value that can be returned by a call to the omp_get_num_threads function
omp_get_thread_num	returns the thread number of the thread, within the team, making this call
omp_get_thread_limit	returns the maximum number of openmp threads available to a program
omp_get_num_procs	returns the number of processors that are available to the program
omp_in_parallel	used to determine if the section of code which is executing is parallel or not
omp_set_dynamic	enables or disables dynamic adjustment (by the run time system) of the number of threads available for execution of parallel regions
omp_get_dynamic	used to determine if dynamic thread adjustment is enabled or not
omp_set_nested	used to enable or disable nested parallelism
omp_get_nested	used to determine if nested parallelism is enabled or not
omp_set_schedule	sets the loop scheduling policy when “runtime” is used as the schedule kind in the openmp directive
omp_get_schedule	returns the loop scheduling policy when “runtime” is used as the schedule kind in the openmp directive
omp_set_max_active_levels	sets the maximum number of nested parallel regions
omp_get_max_active_levels	returns the maximum number of nested parallel regions
omp_get_level	returns the current level of nested parallel regions

omp_get_ancestor_thread_num	returns, for a given nested level of the current thread, the thread number of ancestor thread
omp_get_team_size	returns, for a given nested level of the current thread, the size of the thread team
omp_get_active_level	returns the number of nested, active parallel regions enclosing the task that contains the call
omp_init_lock	initializes a lock associated with the lock variable
omp_destroy_lock	disassociates the given lock variable from any locks
omp_in_final	returns true if the routine is executed in the final task region; otherwise it returns false
omp_set_lock	acquires ownership of a lock
omp_unset_lock	releases a lock
omp_test_lock	attempts to set a lock, but does not block if the lock is unavailable
omp_init_nest_lock	initializes a nested lock associated with the lock variable
omp_destroy_nest_lock	disassociates the given nested lock variable from any locks
omp_set_nest_lock	acquires ownership of a nested lock
omp_unset_nest_lock	releases a nested lock
omp_test_nest_lock	attempts to set a nested lock, but does not block if the lock is unavailable
omp_get_wtime	provides a portable wall clock timing routine
omp_get_wtick	returns a double-precision floating point value equal to the number of seconds between successive clock ticks

Common OpenMP Directives

Directive	Purpose
#pragma omp parallel	Start a parallel region
#pragma omp for	Parallelize a loop
#pragma omp sections	Execute independent sections in parallel
#pragma omp single	Only one thread executes
#pragma omp critical	Protect critical sections
#pragma omp barrier	Synchronize threads

Shared Memory Model:

OpenMP operates on the shared-memory model, where multiple threads can access a common memory space. This simplifies data sharing between threads compared to distributed memory models.

Compiler Directives (Pragmas):

OpenMP primarily uses compiler directives (e.g., `#pragma omp` in C/C++) to specify parallel regions and guide the compiler on how to parallelize code. These directives instruct the compiler to generate multi-threaded code.

Thread Creation and Management:

OpenMP handles the creation, management, and synchronization of threads. A master thread typically initiates the parallel region, and slave threads are forked to execute work concurrently.

Work-Sharing Constructs:

OpenMP provides constructs like `for` loops and `sections` to distribute work among threads, allowing different iterations of a loop or different code sections to be executed in parallel.

Data Environment Management:

OpenMP offers mechanisms to control the scope of variables within parallel regions, allowing variables to be declared as `shared` (accessible by all threads) or `private` (each thread has its own copy).

Synchronization:

OpenMP includes constructs for synchronization, such as `critical` sections, `atomic` operations, and `barriers`, to manage access to shared data and prevent race conditions.

Ease of Use:

OpenMP simplifies parallel programming by abstracting away many low-level details of thread management, allowing developers to focus on identifying and parallelizing computationally intensive sections of their code.

Portability:

OpenMP is a standard API, promoting portability across different shared-memory systems and compilers that support the OpenMP specification.

Key Features:

- Uses **compiler directives** (`#pragma omp ...`) to parallelize code.
- Thread-based parallelism.
- Easy to add to existing sequential code.

2.2 Basic Structure

Example – Parallel Hello World:

```
#include <stdio.h>

#include <omp.h>

int main() {

    #pragma omp parallel

    {

        printf("Hello from thread %d\n", omp_get_thread_num());

    }

    return 0;

}
```

```
gcc -fopenmp hello.c -o hello

./hello
```


Program1: Write a OpenMP program to sort an array on n elements using both sequential and parallel mergesort(using Section). Record the difference in execution time.

Objective :

CO Statement

BTL:

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>
#define SIZE 100000
// ----- MERGE FUNCTION -----
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));
    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    i = 0; j = 0; k = left;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
    free(L);
    free(R);
}
// ----- SERIAL MERGE SORT -----
void serialMergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        serialMergeSort(arr, left, mid);
```

```
        serialMergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
// ----- PARALLEL MERGE SORT -----
void parallelMergeSort(int arr[], int left, int right, int depth) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        if (depth <= 4) {
            #pragma omp parallel sections
            {
                #pragma omp section
                parallelMergeSort(arr, left, mid, depth + 1);
                #pragma omp section
                parallelMergeSort(arr, mid + 1, right, depth + 1);
            }
        }
    }
    else
    {
        // Switch to serial to avoid too many threads
        serialMergeSort(arr, left, mid);
        serialMergeSort(arr, mid + 1, right);
    }
    merge(arr, left, mid, right);
}
// ----- MAIN FUNCTION -----
int main() {
    int *arr_serial = (int *)malloc(SIZE * sizeof(int));
    int *arr_parallel = (int *)malloc(SIZE * sizeof(int));

    // Initialize both arrays with the same random values
    for (int i = 0; i < SIZE; i++) {
        int val = rand() % 100000;
        arr_serial[i] = val;

        arr_parallel[i] = val;
    }
    // ----- SERIAL MERGE SORT -----
    clock_t start_serial = clock();
    serialMergeSort(arr_serial, 0, SIZE - 1);
    clock_t end_serial = clock(); // Fixed variable name
    double time_serial = (double)(end_serial - start_serial) / CLOCKS_PER_SEC;
    // ----- PARALLEL MERGE SORT -----
    clock_t start_parallel = clock();
    parallelMergeSort(arr_parallel, 0, SIZE - 1, 0);
    clock_t end_parallel = clock(); // Fixed variable name
```

```
double time_parallel = (double)(end_parallel - start_parallel) / CLOCKS_PER_SEC;
// ----- OUTPUT -----
printf("Serial Merge Sort Time: %.6f seconds\n", time_serial);
printf("Parallel Merge Sort Time: %.6f seconds\n", time_parallel);
// Optional: Verify correctness
/*
for (int i = 0; i < SIZE; i++) {
    if (arr_serial[i] != arr_parallel[i]) {
        printf("Mismatch at index %d: %d vs %d\n", i, arr_serial[i], arr_parallel[i]);
        break;
    }
}
free(arr_serial);
free(arr_parallel);
return 0;
}
```

OUTPUT:**Case-1**

```
#define SIZE 1000
```

```
gcc -fopenmp program1.c -o pg1
```

```
../ pg1
```

Case-2

```
#define SIZE 10000
```

```
gcc -fopenmp program1.c -o pg1
```

```
../ pg1
```

Case-3

```
#define SIZE 100000
```

```
gcc -fopenmp program1.c -o pg1
```

```
../ pg1
```

Case-4

```
#define SIZE 1000000  
gcc -fopenmp program1.c -o pg1  
../ pg1
```

Library used:

Logic of program:

Outcome:

Program2: Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread.

For example, if there are two threads and four iterations, the output might be the following:

Thread 0 : Iterations 0 -- 1

Thread 1 : Iterations 2 – 3

Objective :

CO Statement

BTL:

Program

```
#include<stdio.h>
#include<omp.h>
int main()
{
    int n = 16,thread,i;
    printf("\n Enter the number of tasks");
    scanf("%d",&n);
    printf("\n n Enter the number of threads");
    scanf("%d",&thread);
    omp_set_num_threads(thread);
    printf("\n ----- \n");
    #pragma omp parallel for schedule(static, 2)
    for (i = 0; i < n; i++)
    {
        printf("Thread %d executes iteration %d\n", omp_get_thread_num(), i);
    }
    return 0;
}
```

Output**Case-1**

```
gcc -fopenmp program2.c -o pg2
./pg2
```

Case-1

```
gcc -fopenmp program2.c -o pg2
./pg2
```

Library used:

Logic of program:

Outcome:

Program3: Write a OpenMP program to calculate n Fibonacci numbers using tasks.

Objective :

CO Statement

BTL:

Program

```
#include <stdio.h>
#include <omp.h>
#include <time.h>
// Serial Fibonacci calculation function
int ser_fib(long int n)
{
    if (n < 2) return n;
    return ser_fib(n - 1) + ser_fib(n - 2);
}

// Parallel Fibonacci calculation function
int fib(long int n)
{
    if (n < 2) return n;

    int x, y;

    #pragma omp task shared(x)
    x = fib(n - 1);

    #pragma omp task shared(y)
    y = fib(n - 2);

    #pragma omp taskwait
    return x + y;
}

int main()
{
    long int n = 10;
    int result;
    clock_t start, end;
    double cpu_time
```

```
printf("Enter the value of n: ");
scanf("%ld", &n);

// Parallel Fibonacci
start = clock();
#pragma omp parallel
{
    #pragma omp single
    result = fib(n);
}
end = clock();
cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nParallel Fibonacci(%ld) = %d", n, result);
printf("\nTime used in parallel mode: %f seconds\n", cpu_time);

// Serial Fibonacci
start = clock();
result = ser_fib(n);
end = clock();
cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nSerial Fibonacci(%ld) = %d", n, result);
printf("\nTime used in serial mode: %f seconds\n", cpu_time);
return 0;
}
```

Output

Case-1

```
gcc -fopenmp program3.c -o pg3
./ pg3
```

Library used:

Logic of program:

Outcome:

Program 4: Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.

Objective :

CO Statement

BTL:

Program

```
#include <stdio.h>
#include <omp.h>
#include <time.h>
int is_prime(int n)
{
    if (n < 2)
        return 0;
    for (int j = 2; j * j <= n; j++)
    {
        if (n % j == 0)
            return 0;
    }
    return 1;
}
int main()
{
    long n = 100000000;
    clock_t start, end;
    double cpu_time;
    printf("\nThe range of numbers is 1 to %ld\n", n);
    printf("\n-----\n");

    // Serial Execution
    start = clock();
    for (i = 0; i < n; i++)
    {
        is_prime(i);
    }
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Time to compute prime numbers serially: %f seconds\n", cpu_time);
    // Parallel Execution
    start = clock();
    #pragma omp parallel for
    for (i = 0; i < n; i++)
    {
```

```
        is_prime(i);
    }
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Time to compute prime numbers in parallel: %f seconds\n", cpu_time);

    return 0;
}
```

Output1**Case 1**

```
gcc -fopenmp program4_primenumber.c -o pg4
./ pg4
```

Output2**Case 1**

```
gcc -fopenmp program4_primenumber.c -o pg4
./ pg4
```

Output3**Case 1**

```
gcc -fopenmp program4_primenumber.c -o pg4  
./pg4
```

Library used:

Logic of program:

Outcome:

Part III – MPI Basics

3.1 Introduction

MPI (**M**essage **P**assing **I**nterface) is a standard for parallel programming on **distributed-memory systems** like clusters.

Key Features:

- Multiple processes (not threads).
- Explicit communication using send and receive.
- Portable across platforms.

3.2 Basic Structure

Example – Parallel Hello World:

```
#include <mpi.h>
```

```
#include <stdio.h>
```

```
int main(int argc, char** argv) {
```

```
    MPI_Init(&argc, &argv);
```

```
    int rank, size;
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Process ID
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Total processes
```

```
    printf("Hello from process %d of %d\n", rank, size);
```

```
    MPI_Finalize();
```

```
    return 0;
```

```
}
```

Compile and run:

```
mpicc hello_mpi.c -o hello_mpi
```

```
mpirun -np 4 ./hello_mpi
```

3.3 Common MPI Functions

Function	Purpose
MPI_Init	Initialize MPI
MPI_Finalize	End MPI program
MPI_Comm_rank	Get process ID

MPI_Comm_size	Get number of processes
MPI_Send	Send data
MPI_Recv	Receive data
MPI_Bcast	Broadcast data to all processes
MPI_Reduce	Combine values from all processes

Sample hello program in mpi

Library to be installed on ubuntu to run MPI programs

sudo apt install openmpi-bin openmpi-common libopenmpi-dev

Creating a simple program

gedit hello.c

Code:

```
//hello.c

#include <mpi.h>

#include <stdio.h>

int main(int argc, char** argv)

{

    MPI_Init(&argc, &argv);

    int rank;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("Hello from processor %d\n", rank);

    MPI_Finalize();

    return 0;

}
```

Execution

mpicc hello.c -o hello

To observe Results

Case-1:

mpirun ./hello

Output1

Case -2:

Output2

mpirun -np 2 ./hello

Case -3:

Output3

mpirun -np 4 ./hello

Case -4:

Output4

mpirun -np 8./hello

Case -5: mpirun -np 6./hello

Output5

Case -6: mpirun -np 16./hello

Output6

./hello

Either request fewer slots for your application, or make more slots available for use.

A "slot" is the Open MPI term for an allocatable unit where we can launch a process. The number of slots available are defined by the environment in which Open MPI processes are run:

1. Hostfile, via "slots=N" clauses (N defaults to number of processor cores if not provided)
2. The --host command line parameter, via a ":N" suffix on the hostname (N defaults to 1 if not provided)
3. Resource manager (e.g., SLURM, PBS/Torque, LSF, etc.)
4. If none of a hostfile, the --host command line parameter, or an RM is present, Open MPI defaults to the number of processor cores

In all the above cases, if you want Open MPI to default to the number of hardware threads instead of the number of processor cores, use the `--use-hwthread-cpus` option.

Alternatively, you can use the `--oversubscribe` option to ignore the number of available slots when deciding the number of processes to launch.

Type `lscpu` to check no. of cores, thread & socket availability.

Library used:

Logic of program:

Outcome:

Program 5: Write a MPI Program to demonstration of MPI_Send and MPI_Recv.**Objective :****CO Statement****BTL:**

STEP1: Open terminal(ctrl+alt+t) short cut key to open terminal

Type

gedit sr.c**type code as in below**

```
// sr.c

#include <mpi.h>
#include <stdio.h>
#include <string.h>
int main(int argc, char** argv)
{
    int rank, size;
    MPI_Status status;

    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get current process rank and total number of processes
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Process 0 sends a message to all other processes
    if (rank == 0)
    {
        for (int i = 1; i < size; i++)
        {
            char message[100];
            sprintf(message, "Hello from process 0 to process %d", i);
            MPI_Send(message, strlen(message) + 1, MPI_CHAR, i, 0,
                MPI_COMM_WORLD);
            printf("Process 0 sent message to process %d\n", i);
        }
    }
}
```

```
    }  
else  
{  
    // All other processes receive the message from process 0  
    char recv_buf[100];  
    MPI_Recv(recv_buf, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD,  
            &status);  
    printf("Process %d received message: %s\n", rank, recv_buf);  
}  
  
// Finalize MPI  
MPI_Finalize();  
return 0;  
}
```

Now For Compilation

mpicc sr.c -o sr

For execution

Case-1

Output 1

Case-2**Output**

mpirun -oversubscribe --np 6 ./sr

Case-3:

Output

`mpirun -oversubscribe --np 16 ./sr`

Program 6: Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence

Objective :

CO Statement

BTL:

STEP1: Open terminal

Type

gedit deadlock.c

type code as in below

```
// deadlock.c

#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv)
{
    int rank, size;
    int send_data, recv_data;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0)
    {
        send_data = 100;
        // blocking send&recieved
        MPI_Send(&send_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&recv_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
        MPI_STATUS_IGNORE);
        printf("Process 0 received %d\n", recv_data);
    }
    else if (rank == 1)
    {
        send_data = 200;
        MPI_Send(&send_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD); // blocking send
        MPI_Recv(&recv_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
        MPI_STATUS_IGNORE);
    }
}
```

```
    printf("Process 1 received %d\n", recv_data);  
}  
MPI_Finalize();  
return 0;  
}
```

Now For Compilation

mpicc deadlock.c -o deadlock

For execution**Output****Case-1:**

mpirun --np 2 ./deadlock

Case-2:**Output**

mpirun --np 8 ./ex_6

Library used:

Logic of program:

Outcome:

Program 7: Write a MPI Program to demonstration of Broadcast operation.**STEP1: Open terminal****Objective :****CO Statement****BTL:****Type****gedit broadcast.c**

type code as in below

```
// broadcast.c
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv)
{
    int rank, size;
    int number;
    // Initialize MPI environment
    MPI_Init(&argc, &argv);
    // Get current process rank and total number of processes
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0)
    {
        number = 42; // The value to be broadcasted
        printf("Process 0 is broadcasting number %d\n", number);
    }

    // Broadcast the value of `number` from process 0 to all others
    MPI_Bcast(&number, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // All processes print the received number
    printf("Process %d received number %d\n", rank, number);
```

```
// Finalize MPI
MPI_Finalize();
return 0;
}
```

Now For Compilation

mpicc broadcast.c -o broadcast

For execution

mpirun -oversubscribe --np 4 ./broadcast

Results:

Case-1

Output

mpirun ./ex_7

Case:2

Output

mpirun --np 4 ./ex_7

Library used:

Logic of program:

Outcome:

Program 8: Write a MPI Program demonstration of MPI_Scatter and MPI_Gather

Objective :

CO Statement

BTL:

STEP1: Open terminal

Type

gedit sg.c

type code as in below

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv)
{
    int rank, size;
    int data[100];    // Only used by root
    int recv_value;    // Value received by each process
    int gathered[100]; // Only used by root

    // Initialize MPI
    MPI_Init(&argc, &argv);
    // Get rank and size
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Only rank 0 initializes data
    if (rank == 0)
    {
        for (int i = 0; i < size; i++)
        {
            data[i] = i * 10;
        }
        printf("Process 0 initialized data: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", data[i]);
        }
        printf("\n");
    }
}
```

```
// SCATTER: One value from `data` array is sent to each process
MPI_Scatter(data, 1, MPI_INT, &recv_value, 1, MPI_INT, 0, MPI_COMM_WORLD);
printf("Process %d received value %d from Scatter\n", rank, recv_value);

// Each process modifies its value
recv_value += rank;

// GATHER: Each process sends modified value back to root
MPI_Gather(&recv_value, 1, MPI_INT, gathered, 1, MPI_INT, 0,
MPI_COMM_WORLD);

// Only rank 0 displays gathered results
if (rank == 0)
{
    printf("Process 0 gathered data: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", gathered[i]);
    }
    printf("\n");
}

// Finalize
MPI_Finalize();
return 0;
}
```

Now For Compilation

mpicc sg.c -o sg

For execution

Case-1

Output

mpirun -oversubscribe --np 4 ./sg

Case-2**Output**

```
mpirun --np 7 ./ex_8
```

Case-3**Output**

```
mpirun ./ex_8
```

Library used:

Logic of program:

Outcome:

Program 9: Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD)

Objective :

CO Statement

BTL:

STEP1: Open terminal

Type

gedit pg9.c

type code as in below

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv)
{
    int rank, size;
    int value;
    int sum_result, prod_result, max_result, min_result;
    int all_sum, all_prod, all_max, all_min;

    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get rank and size
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Each process sets its value
    value = rank + 1; // Example: rank 0 = 1, rank 1 = 2, etc.
    printf("Process %d has value %d\n", rank, value);

    // ----- MPI_Reduce (result only at root, rank 0) -----
    MPI_Reduce(&value, &sum_result, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &prod_result, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);
```

```
MPI_Reduce(&value, &max_result, 1, MPI_INT, MPI_MAX, 0,
MPI_COMM_WORLD);
MPI_Reduce(&value, &min_result, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);

if (rank == 0) {
    printf("\n--- Results using MPI_Reduce (only on root) ---\n");
    printf("Sum  = %d\n", sum_result);
    printf("Prod = %d\n", prod_result);
    printf("Max  = %d\n", max_result);
    printf("Min  = %d\n", min_result);
}

// ----- MPI_Allreduce (result sent to all processes) -----
MPI_Allreduce(&value, &all_sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
MPI_Allreduce(&value, &all_prod, 1, MPI_INT, MPI_PROD, MPI_COMM_WORLD);
MPI_Allreduce(&value, &all_max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
MPI_Allreduce(&value, &all_min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);

printf("Process %d - AllReduce: Sum=%d Prod=%d Max=%d Min=%d\n",
    rank, all_sum, all_prod, all_max, all_min);

// Finalize
MPI_Finalize();
return 0;
}
```

Now For Compilation

mpicc pg9.c -o pg9

For execution

Case-1

Output

mpirun ./ex_9

```
mpirun --np 2 ./pg9
```

Output

```
mpirun --np 2 ./ex_9
```

Output

Case-2

```
mpirun --np 6 ./ex_9
```

Library used:

Logic of program:

Outcome: